

# Entrevista Técnica — Estacionamento VagaFácil (desenhe no draw.io)

---

**Como funciona este exercício.** Abaixo está uma **conversa** entre a equipe e o cliente do sistema **Estacionamento VagaFácil** (administrador de um estacionamento). **Leia com atenção** e, a partir do que as pessoas dizem, **desenhe um diagrama de classes** no [draw.io](https://draw.io) (ou [app.diagrams.net](https://app.diagrams.net)). Não é para programar — é para **modelar**.

 **Seu diagrama precisa mostrar os quatro pilares da OO e os três tipos de relacionamento** (associação, agregação e composição). No fim há a **lista do que entregar**, um **lembrete da notação** e um **gabarito** (para o professor).

---

## A entrevista

**Gerente:** Pessoal, vamos organizar o sistema da **Estacionamento VagaFácil**. Vou trazer o cliente pra ele explicar, do jeito dele, o que o sistema precisa ter. Anotem tudo e depois a gente transforma em diagrama.

**Cliente:** No sistema **Estacionamento VagaFácil**, a peça central é **Cliente**. Tem dois tipos: o **ClienteAvulso** (o comum) e o **ClienteMensalista**, que **tem vaga garantida e não paga por hora**. No fundo, os dois são um tipo de **Cliente** — todos têm nome e placa do veículo e compartilham a mesma base.

**Dev sênior:** Entendi. Então tem uma parte **igual pra todos** e uma parte **específica** de cada tipo. Guardem isso — cheira a uma base comum com dois tipos que herdam dela.

**Cliente:** Isso. E o comportamento muda por tipo: olhando o `calcularValor()` de cada um, pro **ClienteAvulso** paga por hora/fração; já pro **ClienteMensalista** não paga por hora (mensalidade fixa). A **mesma operação, resposta diferente conforme o tipo**.

**Analista de qualidade:** Um ponto sério: o estado da vaga (livre/ocupada) só muda por operações; o valor nunca é negativo. Ninguém pode mexer nesses dados **por fora** — tem que ser por **operações controladas**. E tem uma regra que **NÃO pode falhar: não pode ocupar uma vaga que já está ocupada**. Por isso o estado de **Ticket** só anda por operações (aberto → em uso → fechado → pago), nunca "na mão".

**Cliente:** Agora as ligações. Um cliente gera VÁRIOS tickets; cada ticket ocupa UMA vaga.

**Cliente:** Tem uma ligação de **parte-todo que nasce e morre junto: Ticket ♦ Cobrança**. A cobrança nasce dentro do ticket e não existe fora dele. (pense em **composição** — losango cheio ♦)

**Dev sênior:** Não confunda com o outro caso: **Setor** ◇ **Vaga** é só um **agrupamento**.

As vagas existem no setor por conta própria; o setor só as agrupa. Se **Setor** sumir, **Vaga** continua existindo — isso é **agregação**

(losango vazio ◇), bem diferente da composição.

**Gerente:** Fechou. Com isso dá pra desenhar o modelo. Mãos à obra.

## Dicas de leitura (pistas escondidas na conversa)

| O que apareceu na conversa                                                         | Onde isso te leva                                                       |
|------------------------------------------------------------------------------------|-------------------------------------------------------------------------|
| "os dois são um tipo de <b>Cliente</b> (mesma base)"                               | um tipo <b>geral</b> (base) e tipos <b>específicos</b> → <b>herança</b> |
| "a <b>mesma</b> operação <b>calcularValor()</b> , resposta muda por tipo"          | operação <b>redefinida</b> em cada tipo → <b>polimorfismo</b>           |
| "O estado da vaga (livre/ocupada) só muda por operações; o valor nunca é negativo" | dado <b>privado</b> + operações → <b>encapsulamento</b>                 |
| "não pode ocupar uma vaga que já está ocupada"                                     | <b>invariante</b> : o estado muda só por operação que valida            |
| " <b>Ticket</b> ◆ <b>Cobrança</b> (nascem e somem juntos)"                         | <b>composição</b> (losango cheio ◆)                                     |
| " <b>Setor</b> ◇ <b>Vaga</b> (Vaga existe sozinho)"                                | <b>agregação</b> (losango vazio ◇)                                      |
| "um cliente gera VÁRIOS tickets; cada ticket ocupa UMA vaga"                       | <b>associação</b> com <b>multiplicidade</b>                             |

## O que você deve entregar (no draw.io)

Um **diagrama de classes** que contenha, de forma clara:

1. **Abstração** — uma classe **base** que não é criada diretamente (ex.: **Cliente**).
2. **Herança** — **ClienteAvulso** e **ClienteMensalista** ligados à base pelo triângulo vazio →▷.
3. **Encapsulamento** — atributos sensíveis como **privados** (−) e **operações públicas** (+) que os controlam (proteger o valor e o estado da **Ticket**).
4. **Polimorfismo** — a operação **calcularValor()** declarada na base e **redefinida** em cada tipo.
5. **Os três relacionamentos**, cada um com sua **multiplicidade**:
  - **Associação** (linha/seta): um cliente gera VÁRIOS tickets; cada ticket ocupa UMA vaga.
  - **Agregação** (losango **vazio** ◇): **Setor** junta **Vagas** que existem sozinhos.
  - **Composição** (losango **cheio** ◆): **Ticket** é feita de **Cobranças** que nascem e morrem com ela.

## Lembrete de notação (UML)

| Elemento | Como desenhar |
|----------|---------------|
|----------|---------------|

| Elemento          | Como desenhar                                                     |
|-------------------|-------------------------------------------------------------------|
| Classe abstrata   | nome em <i>itálico</i> + « <b>abstract</b> » (não se cria direto) |
| Operação abstrata | em <i>itálico</i> (cada subtipo redefine)                         |
| Visibilidade      | – privado · + público · # protegido                               |
| Herança           | linha com <b>triângulo vazio</b> ➞ apontando para a base          |
| Associação        | linha (com seta), com <b>multiplicidade</b> nas pontas            |
| Agregação         | losango <b>vazio</b> ◇ no lado do "todo"                          |
| Composição        | losango <b>cheio</b> ◆ no lado do "todo"                          |

Multiplicidades: **1** (exatamente um), **0..1** (zero ou um), **\*** (muitos), **1..\*** (um ou mais).

## ✓ Critério de "pronto"

- ☐ Há uma **classe base abstrata** e os dois tipos herdando dela.
- ☐ Nenhum atributo sensível está público; o valor e o estado da **Ticket** são protegidos.
- ☐ A operação **calcularValor()** aparece na base e é **redefinida** em cada tipo.
- ☐ Aparecem os **três** relacionamentos: **associação**, **agregação** (◇) e **composição** (◆).
- ☐ Você consegue **explicar** por que **Ticket** → **Cobrança** é composição e **Setor** → **Vaga** é agregação.
- ☐ Todas as ligações têm **multiplicidade**.

← Briefing do projeto · 📅 Plano de evolução